

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

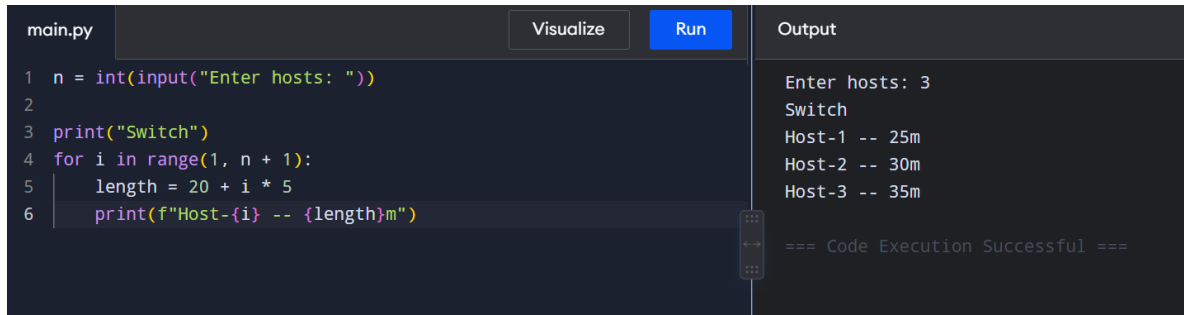
*I **N. HEMA SREE (192511100)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: N. HEMASREE

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function to generate a star topology diagram. It prompts the user to enter the number of hosts, prints the switch name, and then iterates over the hosts to print their labels, the switch name, and the connecting Cat6 UTP cable length. The output shows the results for 3 hosts.

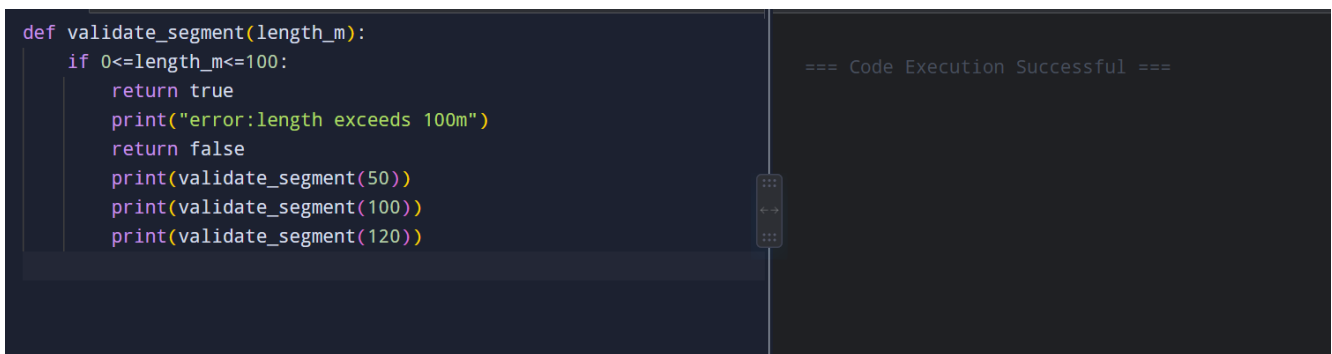
```
main.py Visualize Run Output
```

```
1 n = int(input("Enter hosts: "))
2
3 print("Switch")
4 for i in range(1, n + 1):
5     length = 20 + i * 5
6     print(f"Host-{i} -- {length}m")
```

```
Enter hosts: 3
Switch
Host-1 -- 25m
Host-2 -- 30m
Host-3 -- 35m

=== Code Execution Successful ===
```

2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.

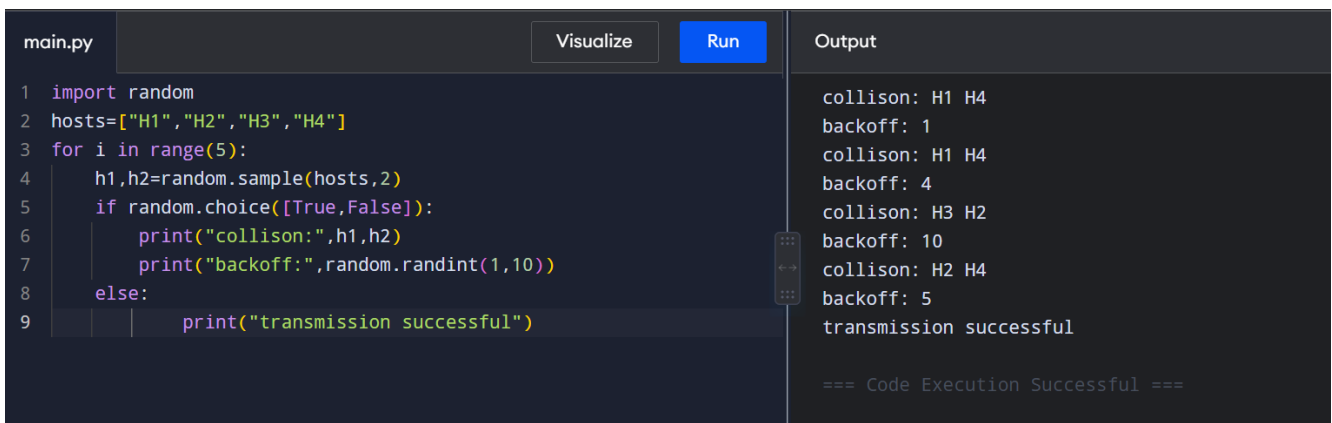


The screenshot shows a Python IDE with a function `validate_segment` that checks if a UTP segment length is within IEEE 802.3 limits (≤ 100 m). The function returns True if valid and False with an error message otherwise. Three test cases are included: 50m, 100m, and 120m.

```
def validate_segment(length_m):
    if 0<=length_m<=100:
        return true
    print("error:length exceeds 100m")
    return false
    print(validate_segment(50))
    print(validate_segment(100))
    print(validate_segment(120))
```

```
=== Code Execution Successful ===
```

3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma_log.txt`.



The screenshot shows a Python IDE with a script that simulates CSMA/CD on a star topology with 4 hosts. The script logs every collision detection event and the back-off interval to a file named `csma_log.txt`. The output shows the results of the simulation.

```
main.py Visualize Run Output
```

```
1 import random
2 hosts=["H1","H2","H3","H4"]
3 for i in range(5):
4     h1,h2=random.sample(hosts,2)
5     if random.choice([True,False]):
6         print("collison:",h1,h2)
7         print("backoff:",random.randint(1,10))
8     else:
9         print("transmission successful")
```

```
collison: H1 H4
backoff: 1
collison: H1 H4
backoff: 4
collison: H3 H2
backoff: 10
collison: H2 H4
backoff: 5
transmission successful

=== Code Execution Successful ===
```

4. Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.

main.py	Visualize	Run	Output
<pre>1 import socket 2 s = socket.socket() 3 s.bind(("localhost", 5000)) 4 s.listen(1) 5 print("Star Switch is running...") 6 c, a = s.accept() 7 msg = c.recv(1024).decode() 8 print("Frame received:", msg) 9 print("MAC Table:") 10 print("H1 -> AA:BB:01") 11 print("H2 -> AA:BB:02") 12 c.send(b"Frame forwarded") 13 c.close()</pre>			Star Switch is running... Frame received: Hello H2 MAC Table: H1->AA:BB:01 H2->AA:BB:02

5. Create a class `StarSwitch` with methods `add\_port(host\_id)`, `remove\_port(host\_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

main.py	Visualize	Run	Output
<pre>1 class StarSwitch: 2 def __init__(self): 3 self.ports = [] 4 def add_port(self, host_id): 5 if host_id not in self.ports: 6 self.ports.append(host_id) 7 print(f"{host_id} added to switch.") 8 def remove_port(self, host_id): 9 if host_id in self.ports: 10 self.ports.remove(host_id) 11 print(f"{host_id} removed from switch.") 12 else: 13 print(f"{host_id} not found.") 14 def broadcast(self, frame):</pre>			Host-1 added to switch. Host-2 added to switch. Host-3 added to switch. Host-4 added to switch. Broadcasting frame... Frame delivered to Host-2: Hello all hosts Frame delivered to Host-3: Hello all hosts Frame delivered to Host-4: Hello all hosts Host-4 removed from switch. === Code Execution Successful ===

